

Module 11 – Application Debugging

1. Introduction to Debugging

What is Debugging?

Debugging is the process of identifying, analyzing, and fixing errors (bugs) in a software application. It helps developers understand how a program executes and why it behaves unexpectedly.

Unlike simply reading code, debugging allows you to pause execution, inspect variables, and trace the program flow in real time.

Why is Debugging Important?

Debugging helps developers to:

- Identify runtime errors.
- Verify program logic.
- Locate incorrect variable values.
- Improve application reliability.
- Reduce development and maintenance time.
- Ensure the application behaves as expected.

Common Types of Bugs

Type	Description
Syntax Error	Errors caused by incorrect language syntax.
Logical Error	Program runs but produces incorrect results.
Runtime Error	Errors that occur while the application is executing.
Performance Issue	Application runs slowly or consumes excessive resources.

2. Debugging in Visual Studio / Browser DevTools

Although different IDEs provide their own debugging interfaces, the debugging concepts remain the same.

Breakpoints

What is a Breakpoint?

A **breakpoint** is a marker placed on a line of code that pauses program execution when that line is reached.

Once execution is paused, developers can inspect variables, evaluate expressions, and determine whether the program is behaving correctly.

Breakpoints eliminate the need for excessive `console.log()` statements and make debugging more efficient.

Placing Breakpoints

Breakpoints can be added by:

- Clicking beside the line number in the editor.
- Pressing the debugger shortcut (commonly **F9** in Visual Studio).
- Adding breakpoints directly within Chrome DevTools.

When execution reaches the breakpoint, the debugger pauses automatically.

Types of Breakpoints

Standard Breakpoint

Stops execution every time the specified line is reached.

Conditional Breakpoint

Stops execution only when a specified condition evaluates to **true**.

Example:

```
count > 100
```

Useful when debugging loops or repeated function calls.

Logpoint

Instead of pausing execution, a logpoint prints information to the console.

Useful when debugging production-like scenarios without interrupting execution.

Inspecting Variables

After the debugger pauses execution, developers can inspect the application's current state.

Watch Window

The **Watch Window** allows specific variables or expressions to be monitored throughout debugging.

Example:

```
totalPrice  
user.name  
cart.length
```

Their values update automatically as execution progresses.

Locals Window

The **Locals Window** displays all variables that are currently available within the active function or scope.

It helps developers quickly examine local variables without manually adding them to the Watch Window.

Hover Inspection

Most modern debuggers allow developers to hover the mouse over a variable to view its current value instantly.

Stepping Through Code

Rather than running the entire application, a debugger allows developers to execute code one statement at a time.

Step Into

Purpose: Enter the called function and execute it line by line.

Example:

```
calculateTotal();
```

Using **Step Into** moves execution inside the `calculateTotal()` function.

Useful when investigating problems inside a function.

Step Over

Purpose: Execute the current line without entering called functions.

The function executes completely, and execution pauses at the next line.

Useful when the called function is already known to work correctly.

Step Out

Purpose: Finish executing the current function and return to the calling function.

Useful when developers accidentally enter a function or have already inspected it.

Continue (Resume)

Resumes program execution until the next breakpoint is encountered or the program finishes.

Typical Debugging Workflow

1. Start the debugger.
2. Place breakpoints.
3. Run the application.
4. Execution pauses at a breakpoint.
5. Inspect variables.
6. Step through the code.
7. Identify the issue.
8. Modify the code.
9. Run the debugger again to verify the fix.

3. Debugging JavaScript with Chrome DevTools

What is Chrome DevTools?

Chrome DevTools is a built-in set of developer tools available in the Google Chrome browser.

It enables developers to:

- Inspect HTML and CSS.
- Debug JavaScript.
- Monitor network activity.
- Analyze application performance.
- Examine browser storage.
- Debug React applications.

Chrome DevTools can be opened using:

- **F12**
- **Ctrl + Shift + I**
- Right-click → **Inspect**

Sources Panel

The **Sources** panel is the primary area for JavaScript debugging.

It allows developers to:

- View source files.
- Set breakpoints.
- Step through code.
- Inspect variables.
- Evaluate expressions.

Setting Breakpoints in Chrome

Open the JavaScript file in the **Sources** panel and click the line number where execution should pause.

When the application reaches that line, Chrome pauses execution automatically.

Console

The **Console** is used to:

- Execute JavaScript commands interactively.
- View log messages.
- Display runtime errors.
- Test expressions during debugging.

Example:

```
console.log(user);
```

Although useful, excessive logging should not replace proper debugging with breakpoints.

Call Stack

The **Call Stack** shows the sequence of function calls that led to the current execution point.

It helps developers determine:

- Which function called another function.
- Where an error originated.
- The execution path of the application.

Scope Panel

The **Scope** panel displays variables currently available in:

- Local scope.
- Block scope.
- Closure scope.
- Global scope.

It allows developers to monitor variable values while stepping through the program.

4. Debugging React Applications with Chrome DevTools

Setting Up the Environment

Before debugging a React application:

- Install **Node.js**.
- Create or open a React project.
- Start the development server using:

`npm start`

or

`npm run dev`

The application is then accessible in the browser.

Installing React Developer Tools

React applications are easier to debug using the **React Developer Tools** browser extension.

The extension adds two additional DevTools tabs:

- **Components**
- **Profiler**

These provide React-specific debugging capabilities.

Launching the Application

Run the React application locally.

Open it in Google Chrome and launch Chrome DevTools.

Inspecting Elements

The **Elements** panel allows developers to inspect:

- HTML structure.
- CSS styles.
- DOM hierarchy.
- Computed styles.

Changes can be tested directly in the browser without modifying the source code.

Debugging JavaScript in React

React applications use JavaScript extensively.

The debugging process is the same as standard JavaScript debugging:

- Set breakpoints.
- Reload the application.
- Pause execution.
- Inspect variables.
- Step through functions.
- Observe state changes.

Debugging React Components

React Developer Tools provide specialized debugging features.

Components Tab

Displays the complete React component hierarchy.

Developers can inspect:

- Component names.
- Props.
- State.
- Hooks.
- Context values.

This is particularly useful for identifying incorrect data being passed between components.

Editing Props and State

React Developer Tools allow developers to temporarily modify props and state during runtime.

This helps test different application scenarios without changing the source code.

Profiler Tab

The **Profiler** measures React component rendering performance.

It helps identify:

- Frequently re-rendered components.
- Slow rendering operations.
- Performance bottlenecks.

Optimizing unnecessary renders can significantly improve application responsiveness.

Common React Debugging Issues

State Not Updating

Possible causes:

- Direct mutation of state.
- Missing state setter function.
- Asynchronous state updates.

Component Not Re-rendering

Possible causes:

- Incorrect dependency arrays.
- Memoization issues.
- Immutable data violations.

Props Not Received

Possible causes:

- Incorrect prop names.
- Parent component not passing data.
- Typographical errors.

Event Handler Not Executing

Possible causes:

- Incorrect event binding.
- Missing callback function.
- Wrong function reference.

Best Practices for Debugging

- Use breakpoints instead of relying solely on `console.log()`.
- Debug one issue at a time.
- Read error messages carefully before making changes.
- Inspect variable values frequently.
- Use Step Into only when necessary.
- Keep functions small and modular to simplify debugging.
- Leverage React Developer Tools for component-level inspection.
- Profile performance when applications become slow.
- Test fixes thoroughly after resolving an issue.

Quick Revision Points

- **Debugging** is the process of identifying and fixing software bugs.
- A **breakpoint** pauses program execution, allowing inspection of variables and program flow.
- **Standard**, **Conditional**, and **Logpoint** breakpoints serve different debugging needs.
- The **Watch Window** tracks selected variables, while the **Locals Window** displays all variables in the current scope.
- **Step Into** enters a function, **Step Over** executes it without entering, **Step Out** returns to the caller, and **Continue** resumes execution.
- **Chrome DevTools** provides the **Sources**, **Console**, **Call Stack**, and **Scope** panels for JavaScript debugging.
- **React Developer Tools** adds **Components** and **Profiler** tabs to inspect props, state, hooks, context, and rendering performance.
- Common React issues include incorrect state updates, missing props, unnecessary re-renders, and improperly bound event handlers.
- Effective debugging combines breakpoints, variable inspection, call stack analysis, and performance profiling to quickly locate and resolve issues.